



INTERNATIONAL
UNIVERSITY OF SARAJEVO

Faculty of Engineering and Natural Sciences · CS308 Software Engineering

Aktour ViaBalkan

Aktourbosna Management System

Final Project Report

Project Team

Furkan Duman · Dogukan Yurtturk · Eren Çapar · Can Aysen

Instructor: Sr. Asst. Mirza Selimović

Table of Contents

1	Introduction	3
1.1	Objectives	3
1.2	Team Members	3
1.3	Document Structure	3
2	Software Process / Development Methodology	4
2.1	The Incremental Cycle	4
2.2	Why Incremental and Not Waterfall	4
2.3	Tools and Engineering Practices	5
3	Requirements Engineering	5
3.1	Stakeholders and User Classes	5
3.2	Functional Requirements	5
3.3	Non-Functional Requirements	5
3.4	Constraints, Assumptions and Dependencies	6
4	System Modeling (UML Diagrams)	6
4.1	Class Diagram	7
4.2	Use Case Diagram	8
4.3	Sequence Diagram - Create New Tour	9
5	Design and Implementation	10
5.1	Design Patterns Applied	10
5.2	Backend Implementation	10
5.3	Frontend Implementation	11
5.4	Guide Role Views	16
6	Architectural Design	17
6.1	High-Level Architecture	18
6.2	Data Model Relationships	18
6.3	Why This Style	19
6.4	Technology Stack	19
6.5	Deployment and Infrastructure	19

7	Testing	21
7.1	Test Strategy	21
7.2	Test Configuration	22
7.3	Unit Test Coverage by Service	22
7.4	End-to-End Scenarios (selected)	22
7.5	How to Run	23
8	Project Management and Planning	23
8.1	Work Organization	23
8.2	Iteration Plan	23
8.3	Version Control and Workflow	24
8.4	Risks and Mitigation	24
9	Software Evolution / Maintenance	24
9.1	Known Gaps in the Current Prototype	24
10	Considerations	25
10.1	Security	25
10.2	Performance and Scalability	25
10.3	Usability	25
10.4	Ethical and Regional Considerations	25
11	Conclusion	25

1. Introduction

Aktour ViaBalkan (internally, the Aktourbosna Management System) is a desktop application that automates tour planning, cost calculation, expense tracking, and financial reporting for tour operators working in the Balkan region. It is the term project for the CS308 Software Engineering course at the International University of Sarajevo.

Tour companies in the Balkans still rely heavily on manual calculations and generic, off-the-shelf management tools that do not understand region-specific data - Balkan toll tariffs, inter-city route distances, and per-vehicle operational costs. These spreadsheet-driven workflows are slow, error-prone, and disconnected: administrators and guides do not share a single source of truth. Aktour ViaBalkan addresses this gap with a single integrated system that calculates tour costs from real Balkan data, tracks expenses with receipt upload, and presents real-time profitability dashboards.

1.1 Objectives

- Automate dynamic tour cost calculation (fuel, tolls, guide fees) from region-specific data.
- Provide secure, role-based access separating Administrator and Tour Guide capabilities.
- Centralize management of tours, routes, waypoints, vehicles, guides, customers, and expenses.
- Offer administrators real-time financial dashboards for profitability analysis.
- Expose a clean REST API so the system can later be extended with web or mobile clients.

1.2 Team Members

The project was developed by a four-person team:

- Furkan Duman
- Dogukan Yurtturk
- Eren Çapar
- Can Aysen

1.3 Document Structure

The report follows the Software Engineering topics covered in lectures and labs: the development methodology (Section 2), requirements engineering (Section 3), system modeling with UML (Section 4), design and implementation (Section 5), architectural design (Section 6), testing (Section 7), project management and planning (Section 8), software evolution and maintenance (Section 9), broader considerations (Section 10), and conclusions (Section 11).

2. Software Process / Development Methodology

The team adopted an incremental, iterative development process inspired by agile practice. Rather than attempting a single big-bang delivery, the system was grown feature by feature, each one designed, implemented, integrated, and tested before the next was started. This suited a four-person student team working in parallel on distinct architectural layers.

2.1 The Incremental Cycle

Each iteration moved a feature from the product backlog (tours, guides, vehicles, routes, expense module, dashboard) through refinement, implementation across the JavaFX UI and Spring Boot API, integration with the database, and delivery as a working increment. When all backlog items were complete, the increments were assembled into the system release.

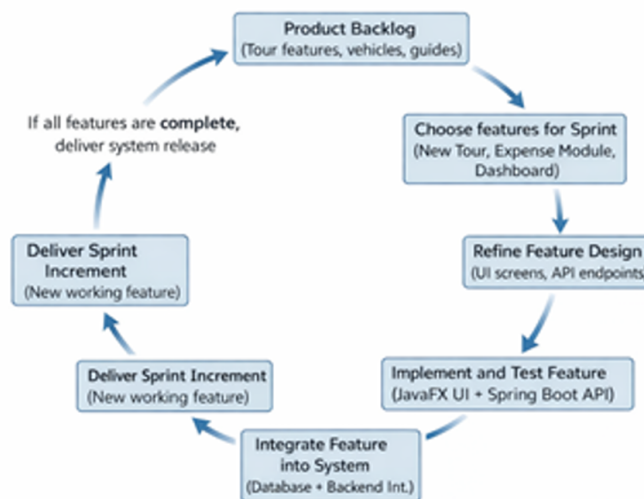


Figure 1. Incremental development cycle adapted for Aktour ViaBalkan.

2.2 Why Incremental and Not Waterfall

- Requirements were refined as the team learned the JavaFX-Spring Boot integration; an upfront waterfall freeze would have been unrealistic.
- Parallel work - frontend, backend, data research, and DevOps proceeded simultaneously on independent modules.
- Each increment produced a demonstrable, testable feature, reducing integration risk late in the term.

2.3 Tools and Engineering Practices

Practice	Tool / Convention
Version control	Git + GitHub, branch-per-feature, 168 commits across 6 contributors
Commit style	Conventional Commits (feat:, fix:, test:, ...)
Build & dependencies	Apache Maven (multi-module: tur-frontend, tur-backend)
IDE / GUI design	IntelliJ IDEA + Scene Builder for FXML
API documentation	SpringDoc OpenAPI (Swagger UI)
Automated testing	JUnit 5, Mockito, Spring Boot Test, MockMvc on a tests branch

3. Requirements Engineering

Requirements were elicited from the project domain (Balkan tour operations), captured in a project proposal, and formalized in an IEEE-style Software Requirements Specification (SRS). Each feature was assigned a priority (High, Medium, Low) to guide iteration planning.

3.1 Stakeholders and User Classes

User Class	Characteristics
System Administrator (Owner)	Creates and manages tours, guides, vehicles, and routes; monitors expenses; analyses financial dashboards. Full access.
Tour Guide	Views assigned tours, uploads receipts, updates tour info, manages own profile. Limited - own data only.
Developer	Responsible for server maintenance, database reliability, and API security.

3.2 Functional Requirements

ID	Requirement	Priority
REQ-1	Secure login mechanism using unique credentials.	High
REQ-2	Role-based access control restricting users to authorized data.	High
REQ-3	Financial charts summarizing tour profitability on the admin dashboard.	High
REQ-4	Dynamic tour cost calculation (fuel, tolls, guide fees) from Balkan data.	High
REQ-5	Tour guides can upload receipts and expense documents.	Medium
REQ-6	Administrators can manage vehicle and guide profiles.	Medium

3.3 Non-Functional Requirements

- **Usability:** a modern, intuitive JavaFX interface that minimizes the learning curve.
- **Security:** role-based access control (RBAC); secured API endpoints; hashed passwords.

- **Performance:** fast API responses with minimal UI delay.
- **Scalability:** a layered architecture that allows new modules and additional client types.

3.4 Constraints, Assumptions and Dependencies

- The frontend must be built with JavaFX and Scene Builder; the backend must expose a Spring Boot REST API.
- Version control via GitHub using Conventional Commits.
- Assumption: accurate Balkan-specific toll and distance data must be maintained for correct cost calculation.
- Dependency: reliable communication between the JavaFX client and the Spring Boot backend over HTTP.

4. System Modeling (UML Diagrams)

Three UML views model the system: a class diagram for static structure, a use case diagram for functional scope per actor, and a sequence diagram for the highest-priority interaction (Create New Tour).

4.1 Class Diagram

Classes were derived from the SRS entities and responsibilities. An abstract **User** base class factors shared authentication fields (username, email, passwordHash, role); **Admin** and **Guide** extend it. **Tour** is the central aggregate: it owns its **Expense** and **Customer** records by composition, follows a **Route**, and uses a **Vehicle** by aggregation. A **Route** owns **Waypoints** (composition) and references shared **Toll** records (aggregation), which carry three tariff categories for cars, minibuses, and coaches.

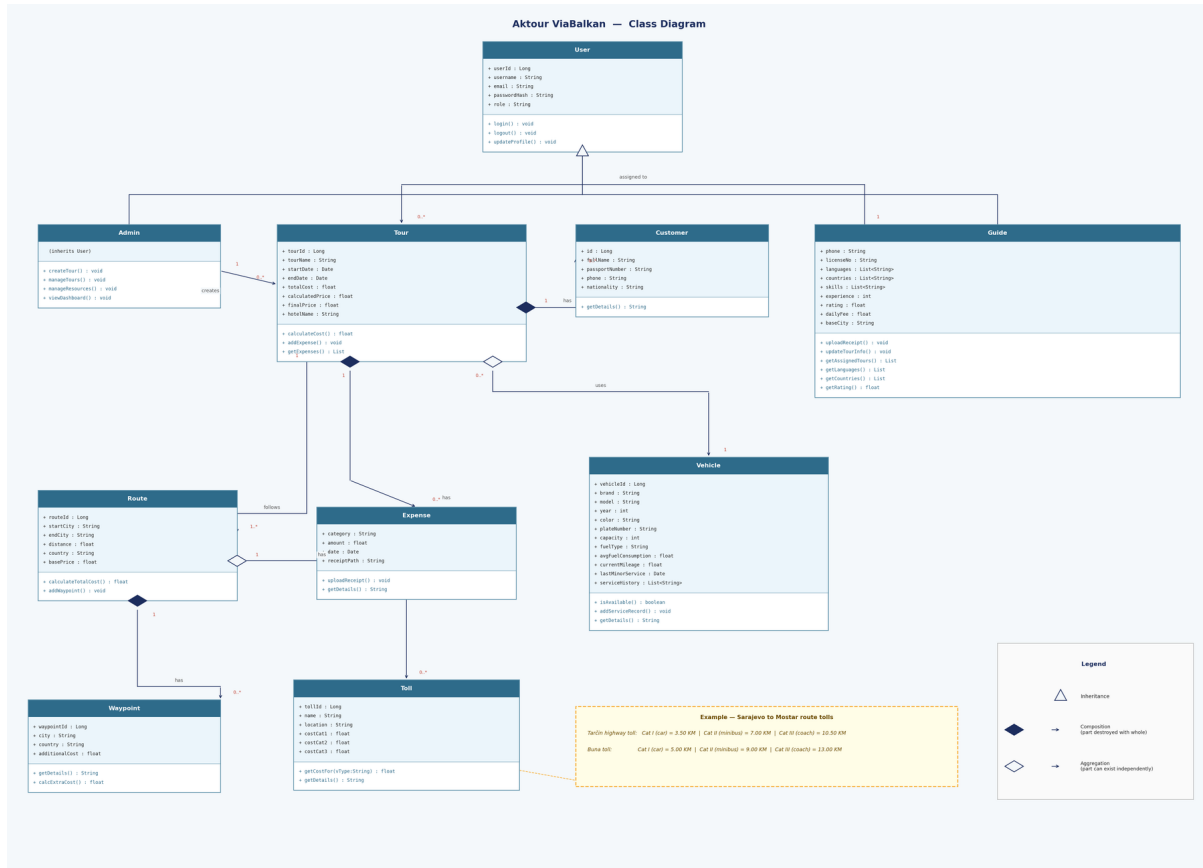


Figure 2. Aktour ViaBalkan - Class Diagram.

4.2 Use Case Diagram

Two actors drive the system. The Admin can log in/out, view the dashboard, create tours, manage guides, vehicles, routes and waypoints, view all expenses, and generate reports. The Guide can log in/out, view assigned tours, update tour info, upload expense receipts, update their own profile, and view their own expenses. The Create New Tour use case «includes» Calculate Tour Cost.

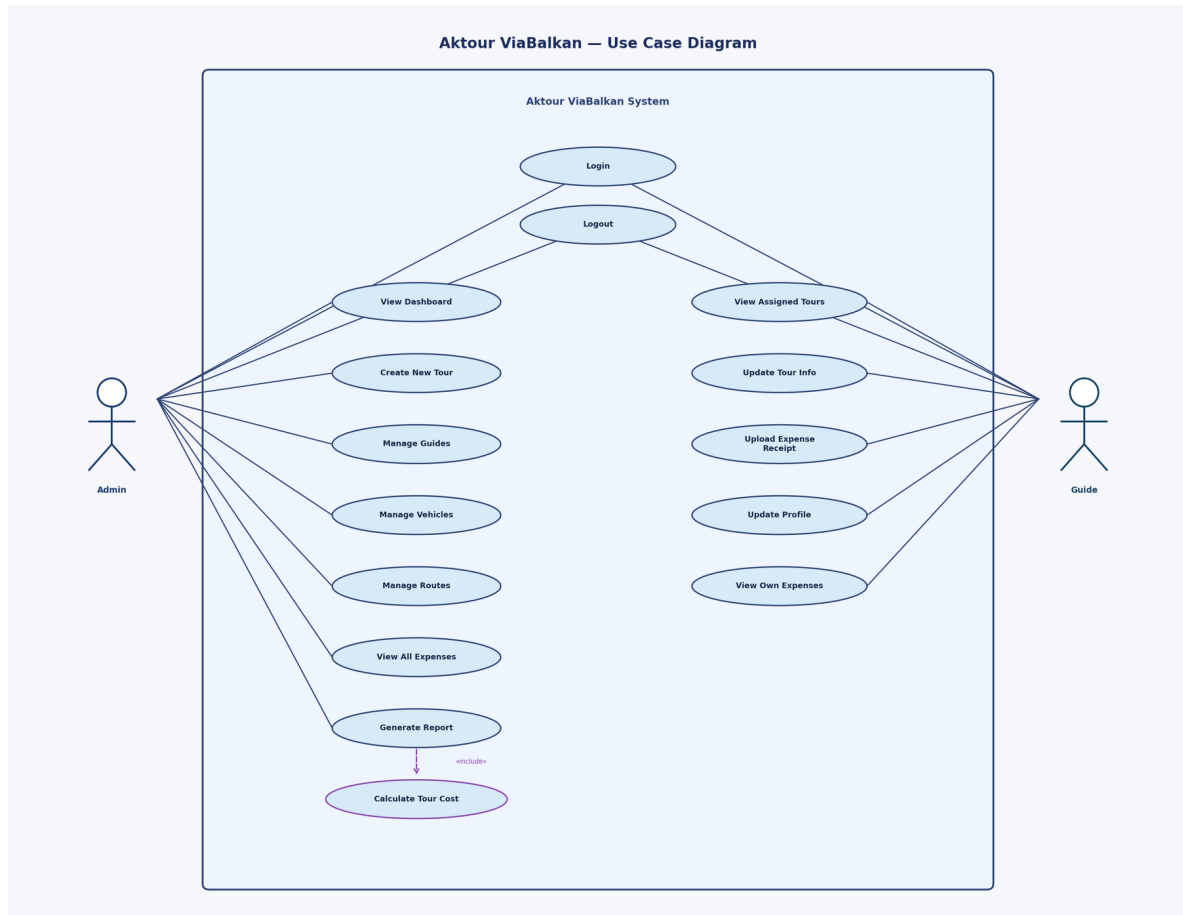


Figure 3. Aktour ViaBalkan - Use Case Diagram.

4.3 Sequence Diagram - Create New Tour

The Create New Tour flow was modeled because it is the highest-priority feature (REQ-4) and exercises all four participants: the Admin, the JavaFX UI, the Spring Boot REST API, and the MySQL/MariaDB database. The UI first fetches routes, guides, and vehicles to populate the wizard, requests a live cost estimate, and finally POSTs the confirmed tour, which the API persists and returns with a generated id.

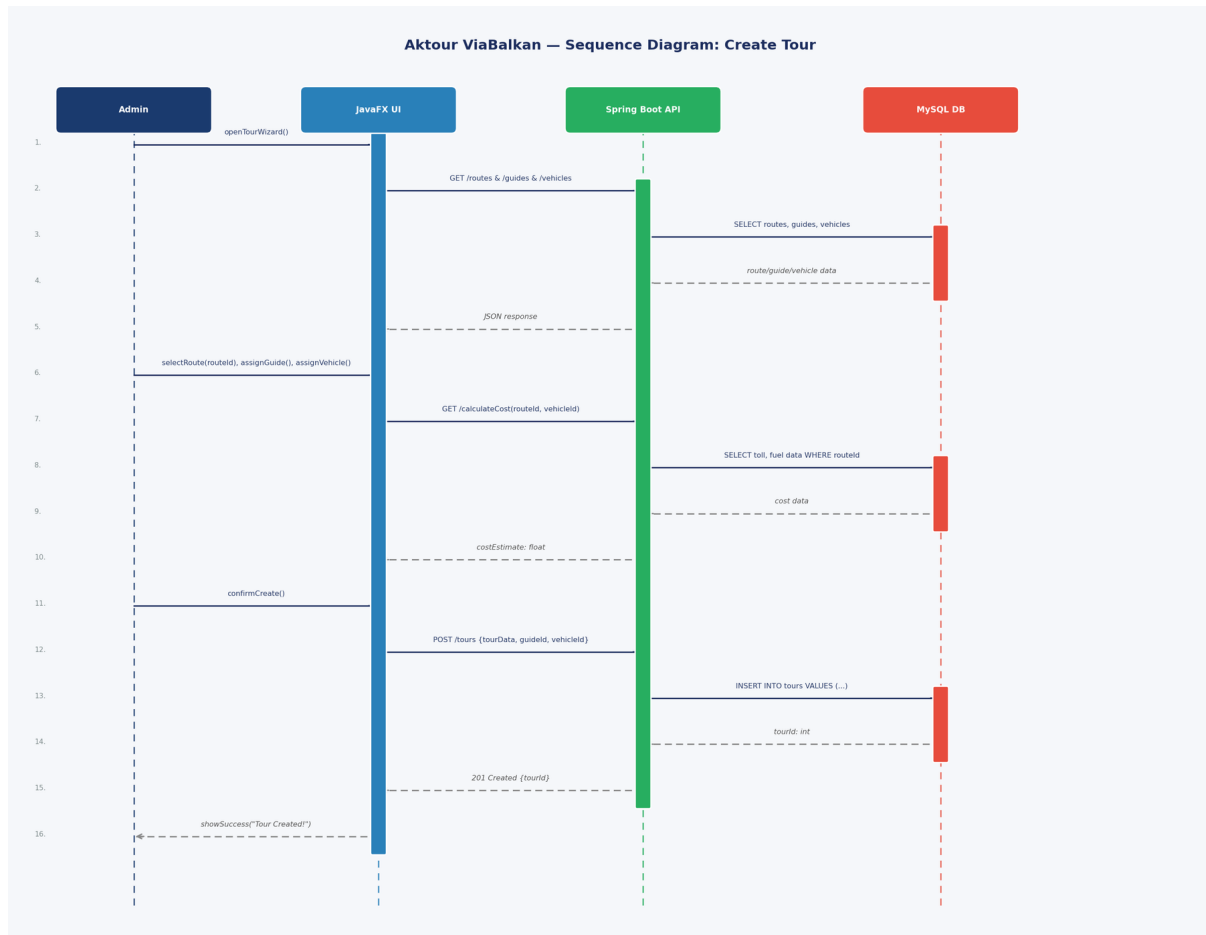


Figure 4. Aktour ViaBalkan - Sequence Diagram for Create New Tour.

5. Design and Implementation

The system is implemented as a Maven multi-module project in Java 21. The **tur-frontend** module is a JavaFX 21 desktop client; the **tur-backend** module is a Spring Boot 3.2.4 REST API backed by a MariaDB database accessed through Spring Data JPA / Hibernate.

5.1 Design Patterns Applied

Pattern	Where & Why
Layered / Service Layer	Controller → DTO/Mapper → Service → Repository. Business logic lives only in services.
Repository	Each <code>*Repository</code> extends <code>JpaRepository</code> ; abstracts all database access behind an interface.
DTO	Request/response DTOs decouple the API from JPA entities; <code>UserResponseDTO</code> omits the password.
Mapper (Map-Struct)	Compile-time, type-safe entity↔DTO mapping; eliminates hand-written copying.
Singleton	Spring-managed beans (controllers, services, repositories) are singletons by default.
Builder	Lombok <code>@Builder</code> / <code>@SuperBuilder</code> on entities avoids telescoping constructors.
Template Method	Abstract <code>User</code> defines login/logout/updateProfile; <code>Admin</code> and <code>Guide</code> specialize.
Facade	Frontend <code>ApiService</code> hides all HTTP/JSON details behind methods like <code>fetchTours()</code> .
Composite	<code>Tour</code> aggregates Customers and Expenses; <code>Route</code> aggregates Waypoints and Tolls.
Dependency Injection	Constructor injection throughout, enabling immutability and easy mock-based testing.

5.2 Backend Implementation

The backend exposes REST resources for users, guides, tours, routes, vehicles, waypoints, expenses, and customers. Controllers parse HTTP and delegate to services; services apply business rules (such as auto-creating a Guide profile when a user with the GUIDE role registers, and computing tour cost from route and vehicle data) within `@Transactional` boundaries; repositories handle persistence. Interactive API documentation is generated automatically by SpringDoc and served at `/swagger` - on the deployed server it is publicly accessible at `aktour.rinnesoft.com/swagger`.

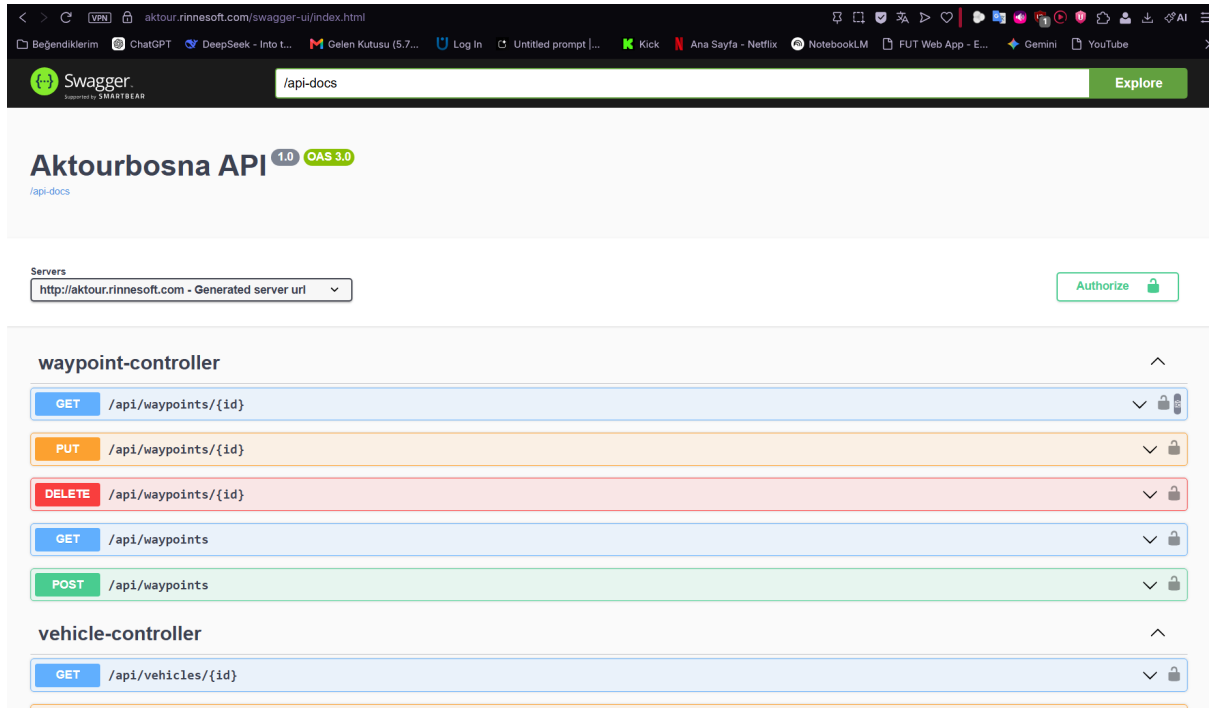


Figure 5. Auto-generated REST API documentation (Swagger UI at aktour.rinnesoft.com/swagger).

5.3 Frontend Implementation

The JavaFX client is organized around FXML views (dashboard, tour/route/vehicle/guide management, expense tracker, guide views) loaded by an **AppController** that handles sidebar navigation. Controllers never speak HTTP directly - they call a single **ApiService** facade that builds requests with `java.net.http` and parses JSON with Gson. The visual design follows a consistent palette and component style across all screens.

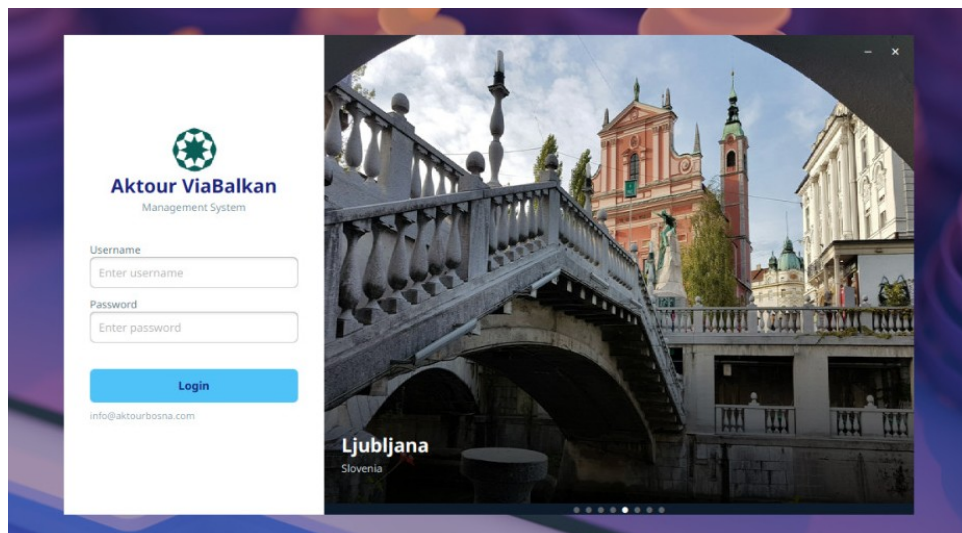


Figure 6. Login screen - entry point for role-based access control.

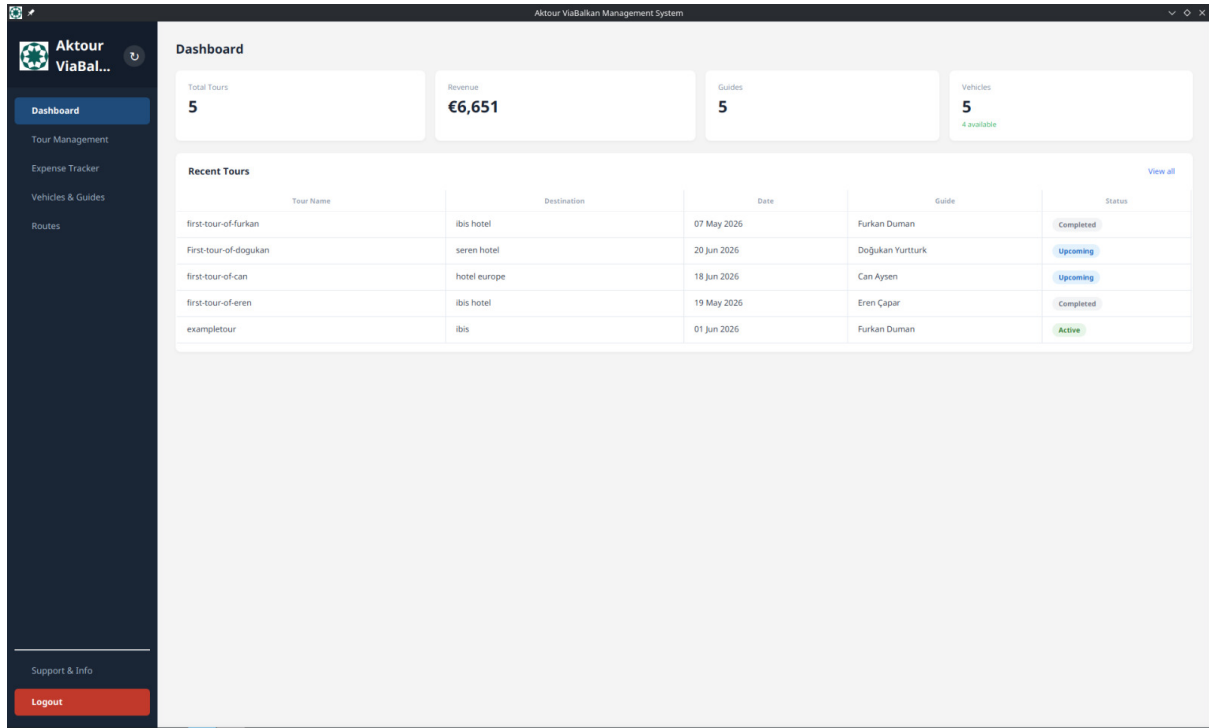


Figure 7. Administrator dashboard with financial charts and KPIs (REQ-3).

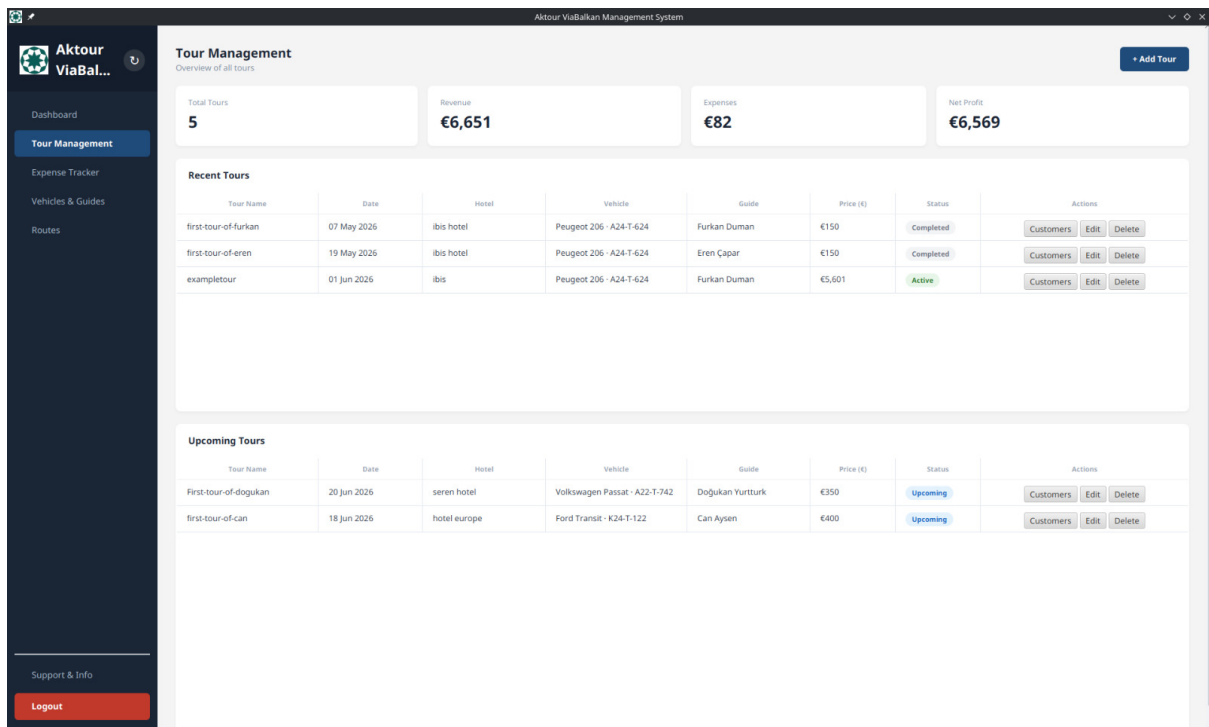


Figure 8. Tour management - list of tours with cost and status.

The 'Add Tour' dialog box contains the following fields and controls:

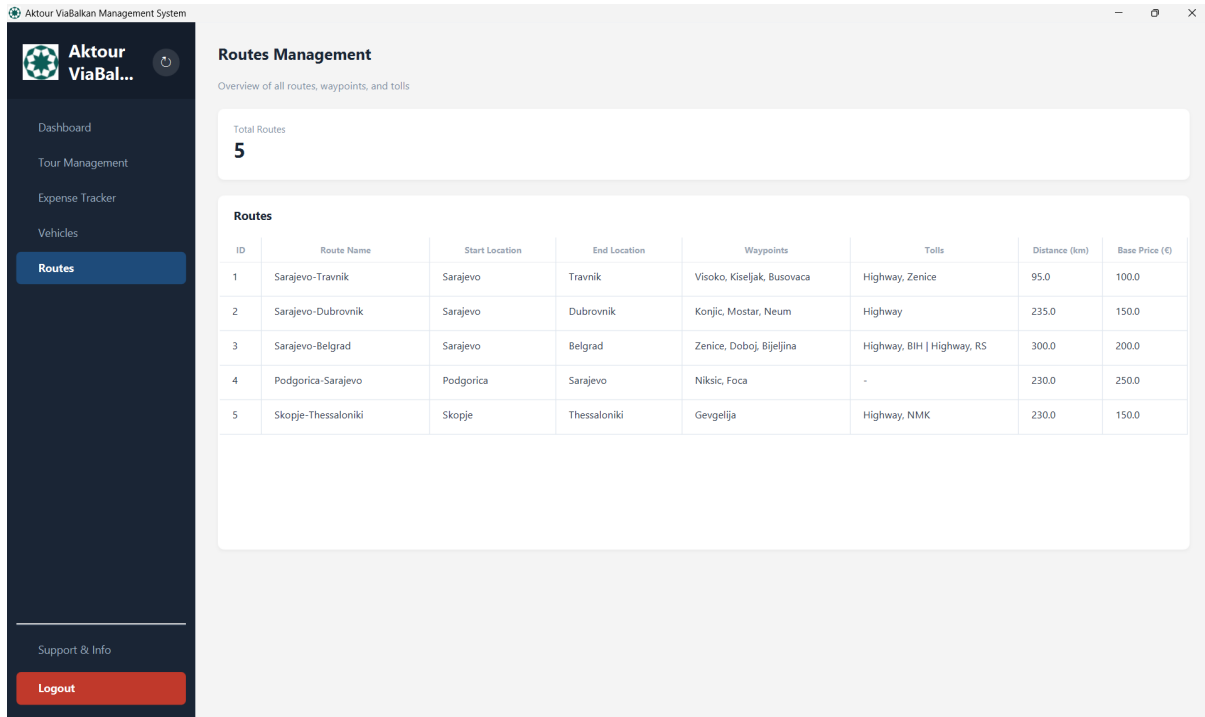
- Tour Name ***: Text input field with placeholder 'Tour name'.
- Start Date ***: Date picker showing '01/06/2026'.
- End Date ***: Date picker showing '08/06/2026'.
- Hotel ***: Text input field with placeholder 'Hotel name'.
- Route ***: Dropdown menu with 'Select route'.
- Vehicle ***: Dropdown menu with 'Select vehicle'.
- Calc. Price (€)**: Text input field with placeholder 'Calculated price (€) — auto from route & vehicle'.
- Final Price (€)**: Text input field with placeholder 'Final price (€)'. Below it, a note reads: 'Doldurulmaması halinde calculated price final price olacaktır.'
- New Customers**: Section header.
- + Add Customer**: Button to add a new customer.
- Save** and **Cancel**: Buttons at the bottom right.

Figure 9. Create Tour dialog - the tour creation wizard with dynamic cost calculation (REQ-4).

The 'Add Tour' dialog box is shown with the 'Add Customer' panel expanded. The fields and controls are the same as in Figure 9, but with additional customer registration options:

- New Customers**: Section header.
- + Add Customer**: Button to add a new customer.
- Customer Registration Panel**: A horizontal row of input fields for: 'Full Name', 'Passport', 'Phone', 'Nationa', and a red ellipsis button '...'.
- Save** and **Cancel**: Buttons at the bottom right.

Figure 10. Create Tour dialog with the Add Customer panel - registering passengers while creating a tour.

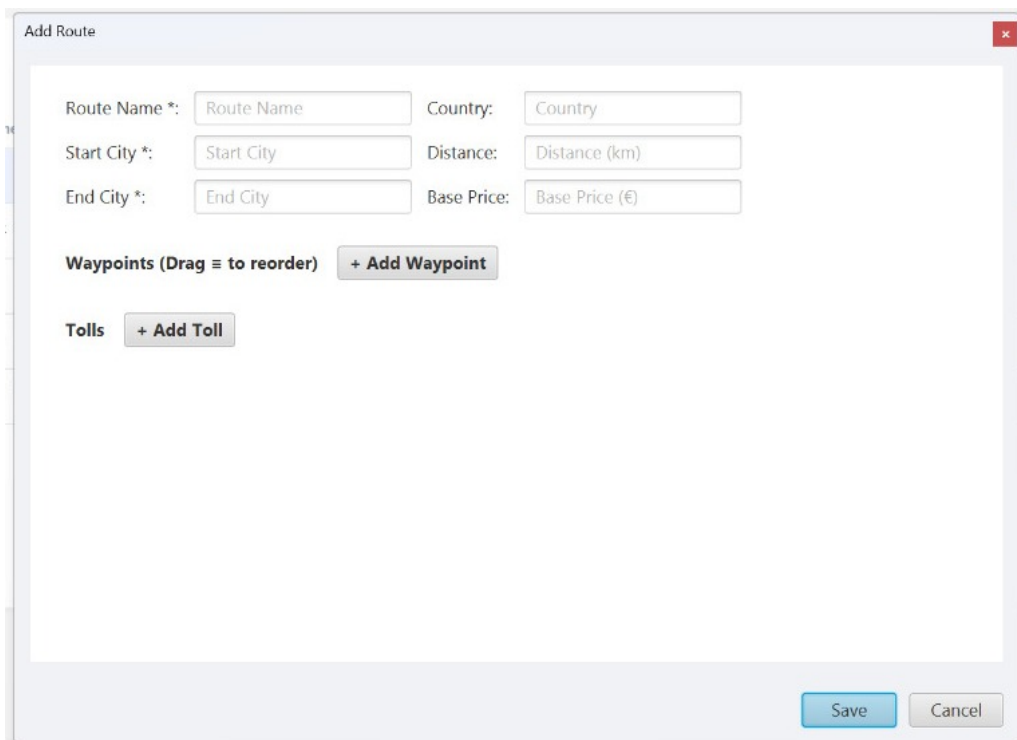


Routes Management
Overview of all routes, waypoints, and tolls

Total Routes
5

ID	Route Name	Start Location	End Location	Waypoints	Tolls	Distance (km)	Base Price (€)
1	Sarajevo-Travnik	Sarajevo	Travnik	Visoko, Kiseľak, Busovaca	Highway, Zenice	95.0	100.0
2	Sarajevo-Dubrovnik	Sarajevo	Dubrovnik	Konjic, Mostar, Neum	Highway	235.0	150.0
3	Sarajevo-Belgrad	Sarajevo	Belgrad	Zenice, Doboj, Bijeljina	Highway, BIH Highway, RS	300.0	200.0
4	Podgorica-Sarajevo	Podgorica	Sarajevo	Niksic, Foca	-	230.0	250.0
5	Skopje-Thessaloniki	Skopje	Thessaloniki	Gevgelija	Highway, NMK	230.0	150.0

Figure 11. Route management - Balkan routes with distance and toll data.



Add Route

Route Name *: Country:

Start City *: Distance:

End City *: Base Price:

Waypoints (Drag ≡ to reorder)

Tolls

Figure 12. Add route dialog - defining start/end cities, distance, waypoints and tolls.

Vehicles & Guides
Overview of all vehicles and guides

Total Vehicles: **5** (4 available) | Total Guides: **5**

Vehicles

ID	Brand	Model	Year	Color	Plate	Seats	Fuel	Price/Day	Status	Actions
1	Ford	Transit	2020	Black	K24-T-122	9	Diesel	€100	Available	Edit Delete
2	Volkswagen	Passat	2015	Grey	A12-K-332	5	Diesel	€45	Not Available	Edit Delete
3	Volkswagen	Passat	2016	Black	A22-T-742	5	Diesel	€45	Available	Edit Delete
4	Peugeot	206	2004	Red	A24-T-624	4	Gasoline	€20	Available	Edit Delete
5	Ford	Mustang	2020	Blue	A24-T-123	5	Electric	€80	Available	Edit Delete

Guides

Username	Email	Phone	Base City	License No	Experience	Daily Fee	Actions
Ahmed Şamil Akçakıl	aakcakil@gmail.com					—	Edit Delete
Furkan Duman	info@rinnsoft.com	+3871111111	Sarajevo		1	—	Edit Delete
Doğukan Yurtturk	dyurtturk@rinnsoft.com					—	Edit Delete
Eren Çapar	ecapar@rinnsoft.com	+1231231123	sarajevo	TRASDASDA	2	€50	Edit Delete
Can Aysen	caysen@rinnsoft.com					—	Edit Delete

Figure 13. Vehicle and guide management screen.

Add New Vehicle

Enter vehicle details

Brand *: e.g. Volkswagen

Model *: e.g. Passat

Year *: e.g. 2020

Color: e.g. Black

Plate Number *: e.g. A24-T-122

Seat Capacity: e.g. 5

Fuel Type *: Diesel

Mileage (km): e.g. 50000

Fuel cons. (l/100km): e.g. 7.5

Daily Fee (€) *: e.g. 100.0

Last Service: Last minor service date

☒ Available

[Save](#) [Cancel](#)

Figure 14. Add vehicle dialog - fleet record with capacity and fuel data.

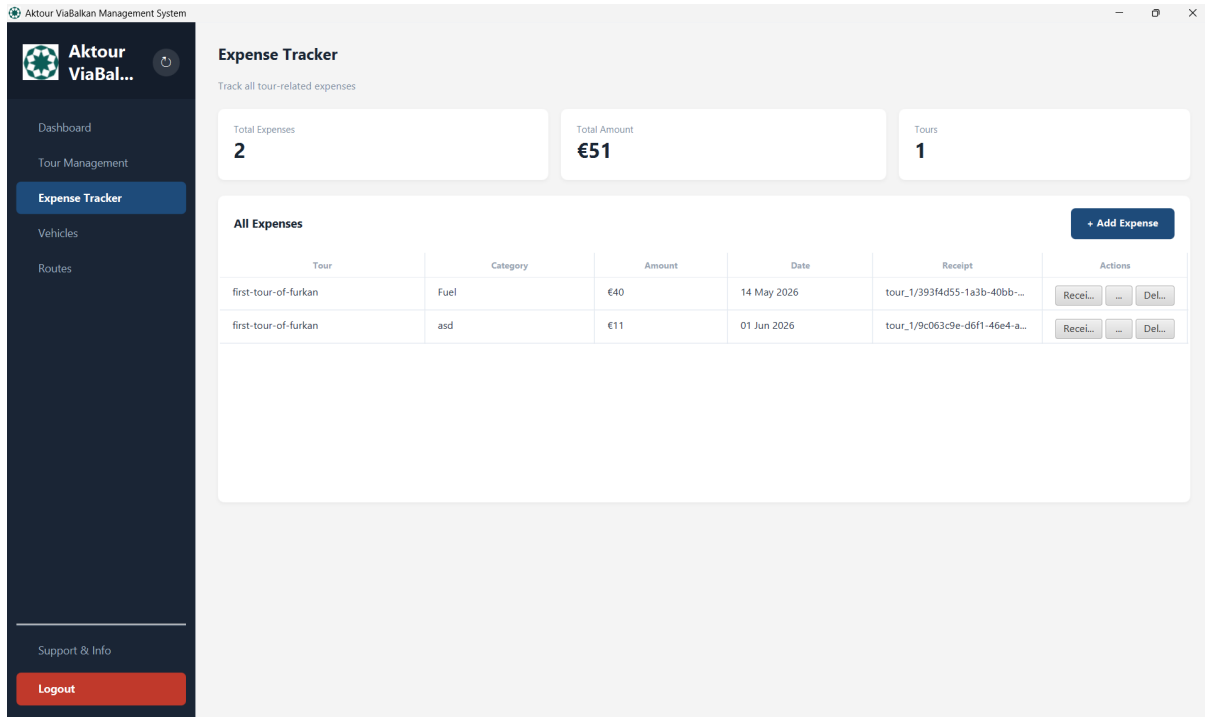


Figure 15. Expense tracker with receipt upload (REQ-5).

The 'Add Expense' dialog box contains the following fields and controls:

- Tour ***: A dropdown menu with the placeholder text 'Select tour'.
- Category ***: A dropdown menu with the placeholder text 'Select category'.
- Amount (€) ***: A text input field with the placeholder text 'Amount (€)'.
- Date**: A date input field showing '1.06.2026' with a calendar icon.
- Receipt**: A text input field with the placeholder text 'Select or type receipt path' and a 'Browse...' button.
- Buttons**: 'Save' and 'Cancel' buttons at the bottom right.

Figure 16. Add expense dialog - logging a cost line-item and attaching a receipt.

5.4 Guide Role Views

Guides see a restricted interface aligned with their RBAC permissions: their own assigned tours and expenses, with no access to system-wide management screens.

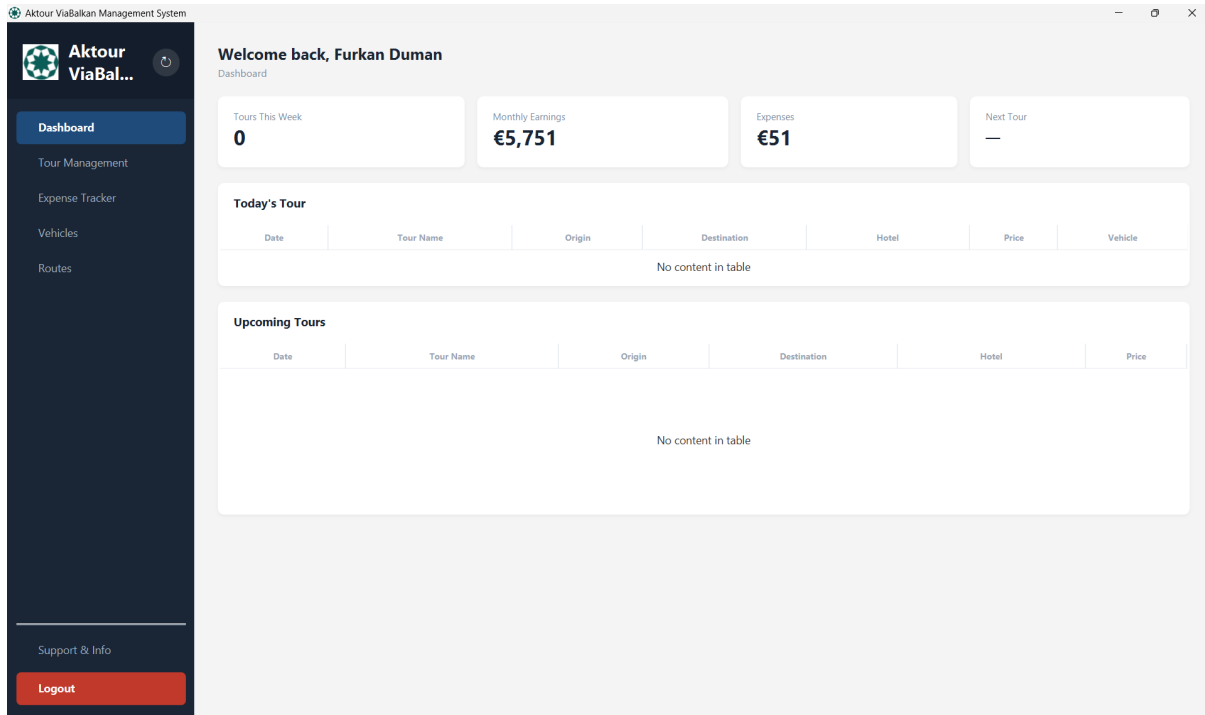


Figure 17. Guide dashboard - restricted view for the Tour Guide role.

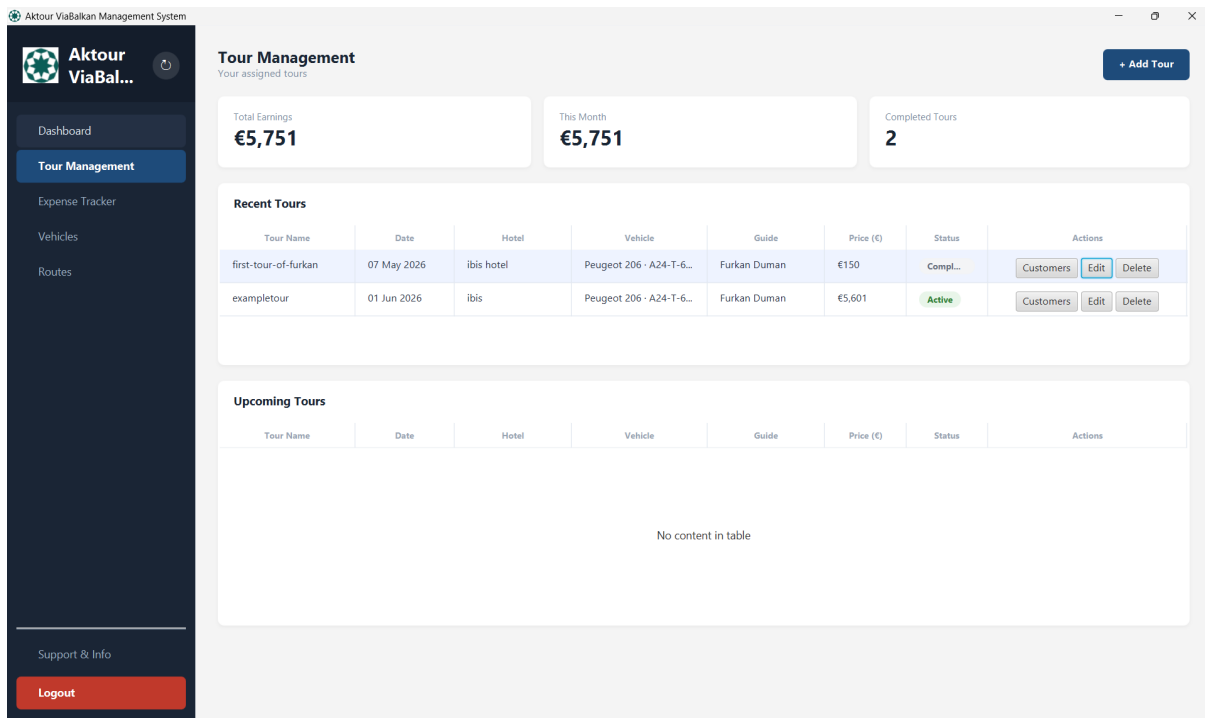


Figure 18. Guide tour management - viewing and updating assigned tours.

6. Architectural Design

Aktour ViaBalkan follows a Client-Server architecture at the macro level, with each side implementing a Layered (N-Tier) structure internally. The JavaFX desktop client and the Spring Boot API are physically and logically separated, communicating only through JSON over

HTTP/REST on port 8080.

6.1 High-Level Architecture

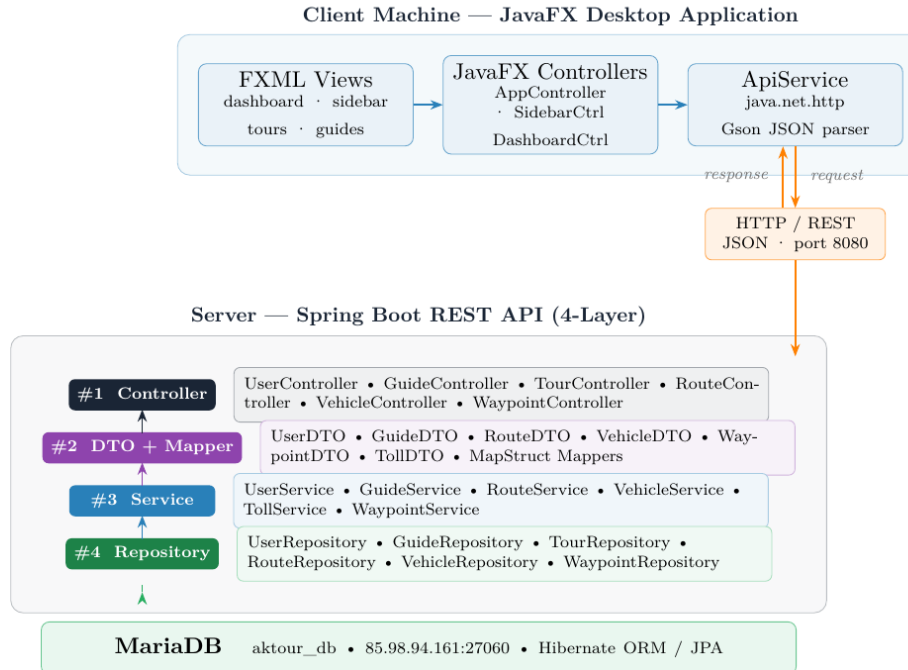


Figure 19. Four-layer layered client-server architecture.

The backend is divided into four layers, each with a single responsibility: the Controller layer parses HTTP; the DTO + Mapper layer translates between API contracts and domain entities; the Service layer holds business logic and transactions; and the Repository layer performs data access via Spring Data JPA. Controllers never touch JPA entities directly, and repositories are the only layer that talks to the database.

6.2 Data Model Relationships

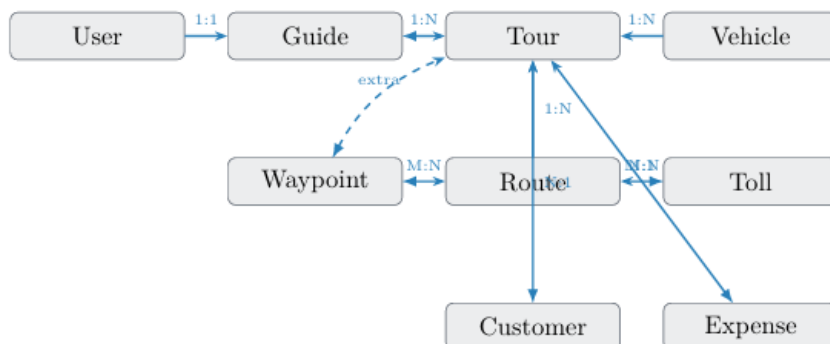


Figure 20. Entity relationship overview.

6.3 Why This Style

- Client-Server allows independent development of UI and business logic, and future reuse of the backend by additional clients.
- Layering localizes change: swapping the database touches only repositories; changing the HTTP client touches only ApiService.
- Microservices were rejected as needless infrastructure overhead for a small academic project; an event-driven style was rejected because the core operations are inherently request-response CRUD.

6.4 Technology Stack

Component	Technology	Version
Backend framework	Spring Boot	3.2.4
Frontend framework	JavaFX	21.0.2
Language	Java	21
ORM	Spring Data JPA + Hibernate	6.x
Database	MariaDB / MySQL	5.7+
Object mapping	MapStruct	1.5.5
Boilerplate	Lombok	1.18.30
API docs	SpringDoc OpenAPI	2.x
JSON (frontend)	Gson	2.10.1
Build tool	Maven	3.x
Security (planned)	Spring Security + JWT	6.x

6.5 Deployment and Infrastructure

The backend and database are not hosted on a cloud provider but on a self-managed physical Linux (Debian) server maintained by the team's DevOps lead. The Spring Boot API and a MariaDB instance run on this machine, which the JavaFX clients reach over the network. This gives the team full control over the environment and demonstrates a complete, real deployment rather than a local-only prototype.



Figure 21. Self-hosted rack server running the backend and database.



Figure 22. Server internals - the physical hardware hosting the Aktour ViaBalkan stack.

The production schema is created and managed by Hibernate on this server. The MariaDB database `aktour_db` contains sixteen tables covering every domain entity - users, guides (with normalized `guide_languages`, `guide_countries`, and `guide_skills` tables), tours, routes, waypoints, `route_waypoints_mapping`, `tour_extra_waypoints`, tolls, `route_tolls`, vehicles, `vehicle_service_history`, customers, and expenses.

```

Linux mkyd 6.12.85-deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.85-1 (2026-04-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Jun 1 10:43:17 2026 from 46.239.2.31
ius2026@mkyd:~$ mysql -u ius2026 -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3130
Server version: 11.8.6-MariaDB-0+deb13u1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> use aktour_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [aktour_db]> show tables;
+-----+
| Tables_in_aktour_db |
+-----+
| customers            |
| expenses             |
| guide_countries      |
| guide_languages      |
| guide_skills         |
| guides              |
| route_tolls          |
| route_waypoints_mapping |
| routes              |
| tolls               |
| tour_extra_waypoints |
| tours               |
| users               |
| vehicle_service_history |
| vehicles            |
| waypoints            |
+-----+
16 rows in set (0.001 sec)

MariaDB [aktour_db]>

```

Figure 23. MariaDB `aktour_db` on the server - `SHOW TABLES` listing the 16 persisted tables.

7. Testing

An automated test suite of 119 tests lives on the dedicated tests branch and targets the `tur-backend` module. It is organized in two layers: 104 unit tests validate service-layer business logic in isolation (repositories, mappers, the file-storage component, and the password encoder replaced with Mockito mocks), and 15 end-to-end tests drive the full HTTP stack through MockMvc against an in-memory H2 database, exercising JSON serialization, Spring Security, JWT authentication, and role-based authorization. The entire suite passes (0 failures, 0 errors).

7.1 Test Strategy

Unit tests follow the Arrange-Act-Assert pattern: collaborators are stubbed, the service method is invoked, the returned DTO is checked with AssertJ, and side effects are confirmed with Mockito `verify(...)`. Every service is covered by both positive (happy-path) and negative (error / not-found) scenarios. The end-to-end `AuthFlowE2ETest` issues real JSON HTTP requests and asserts on status codes and response fields, with a dedicated test Spring profile swapping MariaDB for H2 and seeding administrator accounts.

7.2 Test Configuration

Component	Version / Notes
Test framework	JUnit 5 (Jupiter)
Mocking	Mockito (@Mock, @InjectMocks, MockitoExtension)
Assertions	AssertJ (assertThat, assertThatThrownBy)
Integration	spring-boot-starter-test + MockMvc
Test database	H2 in-memory (MySQL compatibility mode), ddl-auto=create-drop
Security	Spring Security + JWT (jjwt 0.12.5)

7.3 Unit Test Coverage by Service

Test class	Layer	Tests
CustomerServiceTest	Unit	10
ExpenseServiceTest	Unit	10
FileStorageServiceTest	Unit	12
GuideServiceTest	Unit	11
RouteServiceTest	Unit	13
TollServiceTest	Unit	7
TourServiceTest	Unit	13
UserServiceTest	Unit	12
VehicleServiceTest	Unit	9
WaypointServiceTest	Unit	7
Unit subtotal		104
AuthFlowE2ETest	End-to-End	15
Total automated tests		119

7.4 End-to-End Scenarios (selected)

ID	Scenario	Expected
E2E-01	Login with valid admin credentials returns a JWT and role=ADMIN.	200 OK
E2E-02	Login with a wrong password is rejected.	4xx
E2E-04	Protected endpoint accessed with no token.	4xx
E2E-08	Admin creates a new guide (POST /api/guides).	200 OK
E2E-10	Guide token denied access to user management.	403
E2E-13	Request for a non-existing tour (GET /api/tours/999999).	404
E2E-14	Create a customer with a blank fullName (validation).	400
E2E-15	Guide can view their own profile (GET /api/guides/me).	200 OK



8.3 Version Control and Workflow

- GitHub repository with a branch-per-feature workflow and a dedicated tests branch.
- Conventional Commits (feat:, fix:, test:, ...) for a readable, machine-parseable history.
- Maven multi-module layout keeps frontend and backend builds independent yet coordinated.

8.4 Risks and Mitigation

Risk	Mitigation
JavaFX-Spring Boot integration learning curve	Early spike on the ApiService facade isolated networking from the UI.
Accuracy of Balkan toll/distance data	Dedicated data-research role; Toll modeled with three tariff categories.
Integration complexity (frontend ↔ backend)	REST as the sole, well-documented (Swagger) communication channel.
Regression as features grew	119 automated tests on the tests branch guard service logic and security.

9. Software Evolution / Maintenance

The architecture was deliberately shaped for change. Because all business logic is confined to the service layer and all persistence to repositories, the system can evolve along several axes without disturbing unrelated code.

- **New client types:** REST is the sole communication channel, so a web or mobile frontend can be added without backend changes.
- **Database portability:** the Repository pattern hides SQL; the production MariaDB and the H2 test database are swapped purely through configuration.
- **Security hardening:** Spring Security + JWT authentication, BCrypt password hashing, method-level @PreAuthorize RBAC, HTTPS, and a restrictive CORS policy are the planned next iteration, already validated by the end-to-end tests.
- **Data model extension:** guide languages/countries could be normalized into lookup tables, and toll cost fields could carry effective-date ranges for seasonal pricing.
- **Maintainability:** MapStruct, Lombok, DTOs, and constructor injection keep components small, explicit, and individually testable.

9.1 Known Gaps in the Current Prototype

- The shipped prototype operated without an authentication framework; security is layered in on the tests/security branch.
- Some REST endpoints (parts of users, tours, waypoints, expenses) were still in progress at the time of writing and are tracked in the API status table.

10. Considerations

10.1 Security

RBAC distinguishes Admin and Guide. Passwords are hashed with BCrypt and never returned in responses (`UserResponseDTO` omits the field). The intended design issues signed JWTs (HMAC-SHA256, 24-hour expiry) validated on every request, enforces authorization at the method level with `@PreAuthorize`, encrypts transit with HTTPS/TLS, and replaces the permissive development CORS policy with a restrictive one.

10.2 Performance and Scalability

Singleton, stateless services keep memory overhead low and behavior consistent under concurrent requests. The clean client-server split means the backend can serve multiple clients, and the layered design admits caching or read-replicas later without touching business logic.

10.3 Usability

A consistent JavaFX visual language, dialog-driven create/edit flows, and role-tailored views reduce the learning curve for both administrators and guides.

10.4 Ethical and Regional Considerations

Customer records include passport numbers and nationality for Balkan border-crossing documentation; such personal data must be protected, which reinforces the security roadmap above. The system's value rests on accurate, regularly maintained Balkan toll and distance data.

11. Conclusion

Aktour ViaBalkan delivers an integrated, role-aware solution to a real operational problem: the manual, error-prone, region-blind cost and expense management that Balkan tour operators face today. The team applied the full software engineering lifecycle - incremental process, formal requirements, UML modeling, a layered client-server architecture, pattern-driven implementation, and a 119-test automated suite - to produce a working JavaFX + Spring Boot system backed by MariaDB.

The design choices that matter most are the strict separation of concerns (UI → REST → service → repository) and the disciplined use of DTOs, mappers, and dependency injection. Together they make the system testable today and extensible tomorrow, whether the next step is hardening security, adding a web client, or enriching the Balkan data model. The project demonstrates that careful architecture, not just working features, is what lets software survive change.

References

1. Aktour ViaBalkan - Project Proposal, CS308, International University of Sarajevo, March 2026.
2. Software Requirements Specification for Aktour ViaBalkan, Version 1.0, March 2026.
3. Aktour ViaBalkan - System Design Document (UML Class, Use Case, Sequence Diagrams), Version 2.0, May 2026.
4. Aktour ViaBalkan - Architecture Design Documentation, Version 1.0, May 2026.
5. Aktour ViaBalkan - Software Testing Documentation, March 2026.